```
WebApplication app = builder.Build();

app.MapGet("/", () =>"Hello world!");

app.Run();
```

❶ The current IHostEnvironment is available on WebApplicationBuilder.

❷ It’s common to make the base appsettings.json compulsory.

❸ Adds an optional environment-specific JSON file where the filename varies with the environment

With this pattern, a global appsettings.json file contains settings applicable to most environments. Additional optional JSON files called appsettings.Development.json, appsettings.Staging.json, and appsettings.Production.json are subsequently added to `ConfigurationManager`, depending on the current `EnvironmentName`.

Any settings in these files will overwrite values from the global appsettings.json if they have the same key, as you’ve seen previously. Using environment-specific settings files lets you do things like set the logging to be verbose only in the development environment and switch to more selective logs in production.

Another common pattern is to add or remove configuration providers depending on the environment. You might use the User Secrets provider when developing locally, for example, but Azure Key Vault in production. Listing 10.14 shows how you can use `IHostEnvironment` to include the User Secrets provider conditionally only in development. Again, `WebApplicationBuilder` uses this pattern by default.

Listing 10.14 Conditionally including the User Secrets configuration provider

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

IHostEnvironment env = builder.Environment;

builder.Configuration.Sources.Clear();
builder.Configuration
    .AddJsonFile(
```